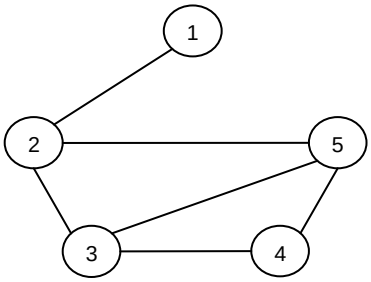
9) Grafi

In questo capitolo vengono illustrati i ***grafi***, importantissime strutture discrete che da un lato esibiscono proprietà di grande interesse per la matematica (in particolare per la teoria dei grafi) e, dall'altro, permettono di modellare una grande varietà di problemi.

9.1 Definizioni

Un ***grafo*** *G = (V, E)* è costituito da una coppia di insiemi, l'insieme finito *V* di ***nodi***, o ***vertici***, e l'insieme finito *E* di coppie non ordinate di nodi, dette ***archi*** o ***spigoli***. Laddove non sorgono equivoci, gli archi vengono generalmente rappresentati come coppie racchiuse tra parentesi tonde – ad es. (u, v) - sebbene, essendo coppie non ordinate, sarebbe più corretto racchiuderle tra parentesi graffe – ad es. {u, v}. Usualmente, gli interi *n* ed *m* indicano rispettivamente il numero di nodi e di archi di cui è composto il grafo.

E' facile vedere che il massimo numero di archi è *n(n - 1)/2*. Nel caso in cui *m = O(n)* diremo che il **grafo** è **sparso**.



V = {1, 2, 3, 4, 5}

E = {(1, 2), (2, 3), (2, 5), (3, 4), (3, 5), (4, 5)}

Un arco *(u, v)* con *u = v* è detto ***cappio***. Un arco che compare più di una volta in *E* è detto ***arco multiplo***. Un ***grafo semplice*** non contiene cappi né archi multipli.

Un ***grafo orientato*** (detto anche ***digrafo***) è definito in modo analogo al grafo, ma gli archi sono delle coppie ordinate e sono detti ***archi orientati***.

Un grafo è detto ***pesato*** se ad ogni arco è associato un valore numerico detto ***peso***.

In questa brevissima trattazione, ometteremo di trattare grafi con cappi e archi multipli, grafi orientati e grafi pesati, e quindi si userà la sola parola *grafo* per intendere la locuzione *grafo semplice non orientato e non pesato*.

Il numero di archi incidenti in un certo nodo *v* è detto ***grado*** *di v (**degree(v)**)*.

Una ***passeggiata*** in *G* dal nodo *x* al nodo *y* è una sequenza $x_1, e_1, x_2, e_2, \ldots, x_t, e_t, x_{t+1}$ in cui si alternano nodi ed archi del grafo con $x_i \in V$, $e_i \in E$, $x = x_1$, $y = x_{t+1}$ e tale che per $i = 1, 2, \ldots, t$ si abbia $\{x_i, x_{i+1}\} = e_i$. La ***lunghezza della passeggiata*** è *t*, ovvero il numero dei suoi archi (incluse le eventuali ripetizioni).

Un ***cammino*** da *x* a *y* è una passeggiata da *x* a *y* in cui non ci siano ripetizioni di archi: se $i \neq j$ allora $e_i \neq e_j$.

Un cammino in cui primo e ultimo nodo coincidono si chiama ***circuito***.

Un ***cammino semplice*** è un cammino in cui non ci siano ripetizioni di nodi: se $i \neq j$ allora $x_i \neq x_j$, ad eccezione al più di *1* e *t + 1*; nel caso in cui $x_1 = x_{t+1}$ il cammino semplice si chiama ***ciclo***.

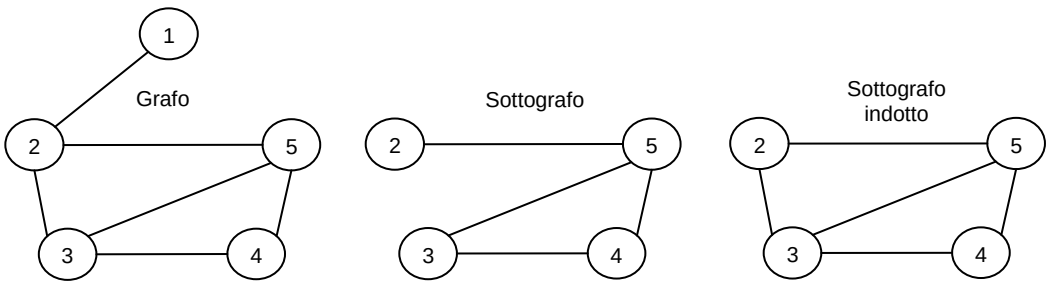
Si dice che *y* è ***raggiungibile*** da *x* se *y = x* oppure se esiste una passeggiata da *x* a *y*.

- La relazione di raggiungibilità tra nodi ha la proprietà ***riflessiva*** per definizione perché *x* è raggiungibile da *x*.
- La relazione è ***simmetrica*** perché se $x_1, e_1, x_2, e_2, \ldots, x_t, e_t, x_{t+1}$ è una passeggiata da *x* ad *y*, allora $x_{t+1}, e_t, x_t, \ldots, x_2, e_1, x_1$ è una passeggiata da *y* ad *x*.
- Inoltre la relazione di raggiungibilità è anche ***transitiva***: se c'è una passeggiata da *x* a *y* e una passeggiata da *y* a *z*, allora *z* è raggiungibile da *x* attraverso la passeggiata che si ottiene concatenando la passeggiata da *x* a *y* con quella da *y* a *z*.

La relazione di raggiungibilità è perciò una relazione di equivalenza: le classi di questa relazione di equivalenza determinano una partizione dei nodi. Tutti i nodi di una stessa classe sono raggiungibili da ogni altro nodo della classe. Ogni classe della partizione induce un sottografo chiamato ***componente connessa***. Un grafo in cui la partizione in questione ha una sola classe si chiama ***connesso***.

Non è restrittivo supporre che i grafi considerati siano connessi: se così non fosse si potrebbero considerare separatamente, una per una, tutte le loro componenti connesse.

Dato un grafo G = (V, E), un **sottografo** di *G* è un grafo $G' = (V', E')$ tale che $V \subseteq V'$ ed $E \subseteq E'$. Dato un sottinsieme *V'* di *V*, si definisce **sottografo indotto** da *V'* il sottografo di *G* che ha come insieme dei vertici l'insieme *V'* e come insieme degli archi l'insieme di ***tutti*** gli archi di *G* che collegano nodi in *V'*. Un **sottografo** $G' = (V', E')$ è **ricoprente** per *G* se $V' = V$.



9.2 Rappresentazione in memoria di grafi

Vi sono diverse tecniche per rappresentare un grafo in memoria:

- liste di adiacenza;
- matrice di adiacenza;
- matrice di incidenza;
- lista di archi.

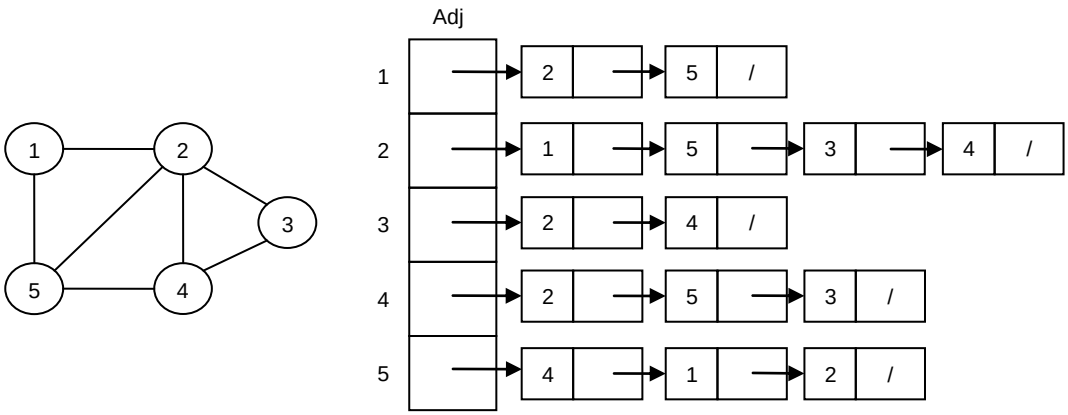
Le prime due tecniche sono quelle più spesso utilizzate in pratica.

Come vedremo nel seguito, e come abbiamo visto nel caso della memorizzazione di alberi, non c'è un modo per memorizzare grafi che risulti migliore in assoluto: è la natura del problema da risolvere che determina la scelta della tecnica di rappresentazione del grafo più idonea alla soluzione.

9.2.1 Liste di adiacenza

Questa memorizzazione consiste in un vettore *Adj[]* di $|V|$ puntatori, uno per ciascun vertice in V, che rappresentano la testa di $|V|$ liste.

Per ciascun vertice $v \in V$, *Adj[v]* punta ad una lista i cui elementi contengono gli indici di tutti i nodi u tali che esiste un arco (u, v) in G. In altre parole, *Adj[v]* punta ad una lista che contiene gli indici di tutti i nodi adiacenti a v in G.



Si noti che un arco (u, v) in un grafo non orientato appare due volte:

- esiste l'indice u nella lista puntata da *Adj[v]*;
- esiste l'indice v nella lista puntata da *Adj[u]*.

Viceversa, nel caso dei grafi orientati l'arco (u, v) uscente da u ed entrante in v appare una sola volta:

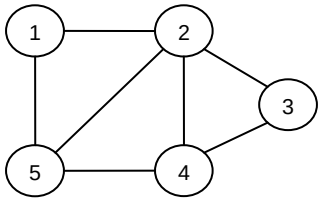
- esiste l'indice *v* nella lista puntata da *Adj[u]*.

Per rappresentare un grafo pesato è sufficiente aggiungere ad ogni elemento delle liste un campo che contiene il peso dell'arco.

9.2.2 Matrice di adiacenza

Questa rappresentazione consiste in una matrice *A* di n^2 numeri interi, nella quale:

- A[i, j] = 1 se e solo se l'arco (i, j) ∈ E(G);
- A[i, j] = 0 altrimenti



	1	2	3	4	5
1		1			1
2	1		1	1	1
3		1		1	
4		1	1		1
5	1	1		1	

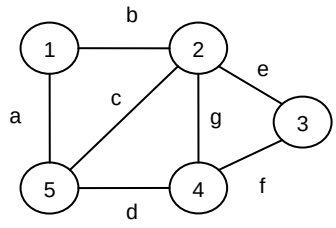
Si noti la simmetria rispetto alla diagonale principale. Essa è dovuta al fatto che, essendo il grafo non orientato, l'arco *(u, v)* e l'arco *(v, u)* coincidono.

Nel caso di grafo orientato la definizione è identica (ma questa volta la coppia che rappresenta l'arco è ordinata!), e quindi la matrice non risulta più simmetrica.

Per rappresentare un grafo pesato è sufficiente memorizzare nella matrice i pesi degli archi anziché il valore 1.

9.2.3 Matrice di incidenza

Questa rappresentazione consiste in una matrice *A* di *n* righe (una per ogni vertice) ed *m* colonne (una per ogni arco), nella quale in ogni colonna vi è un 1 in corrispondenza delle righe relative ai due nodi in cui l'arco in questione è incidente.



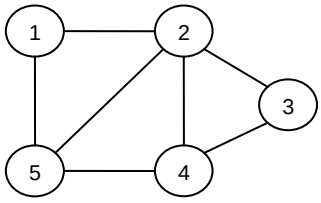
	a	b	c	d	e	f	g
1	1	1					
2		1	1		1		1
3					1	1	
4				1		1	1
5	1		1	1			

Per rappresentare un grafo pesato è sufficiente memorizzare nella matrice i pesi degli archi anziché il valore 1. Per memorizzare un grafo orientato, basta rappresentare in modo differente il nodo di partenza e quello di destinazione di ciascun arco (ad esempio con -1 e 1, rispettivamente).

9.2.4 Lista di archi

Questa rappresentazione consiste semplicemente in un vettore che contiene tutti gli archi del grafo.

Ogni elemento del vettore rappresenta un arco *(u, v)* mediante la coppia degli indici dei nodi *u* e *v* in cui esso è incidente.



1	1	1	2	4	2	3	2
2	5	2	5	5	3	4	4
	1	2	3	4	5	6	7

Per rappresentare un grafo pesato è sufficiente aggiungere ad ogni elemento del vettore un campo che contiene il peso dell'arco. Un grafo orientato verrà rappresentato allo stesso nodo, ma gli archi andranno considerati come orientati dal primo al secondo indice.

9.2.5 Confronti fra le rappresentazioni

In questo paragrafo confrontiamo le varie rappresentazioni di grafi dal punto di vista dell'occupazione di memoria e della complessità di due semplici operazioni.

Occupazione di memoria	
Liste di adiacenza	$\Theta(n + m)$
Matrice di adiacenza	$\Theta(n^2)$
Matrice di incidenza	$\Theta(nm)$
Lista di archi	$\Theta(m)$

Dato il vertice *v*, calcolare *degree(v)*	
Liste di adiacenza	Θ(degree(v)) Si scorre la lista puntata da Adj(v)
Matrice di adiacenza	$\Theta(n)$ Si scorre la v-esima riga della matrice
Matrice di incidenza	$\Theta(m)$ Si scorre la v-esima riga della matrice
Lista di archi	$\Theta(m)$ Si scorre l'intera lista di archi

Dati i vertici *u* e *v*, l'arco (u, v) $\in$ E?	
Liste di adiacenza	Θ(degree(u)) Si scorre la lista puntata da Adj(u)
Matrice di adiacenza	$\Theta(1)$ Si ispeziona l'elemento M[u,v] della matrice
Matrice di incidenza	$\Theta(m)$ Si scorre la u-esima riga della matrice; quando si trova un 1 si verifica se nella nella riga v-esima della stessa colonna è posto un 1
Lista di archi	$\Theta(m)$ Si scorre l'intera lista di archi

Si noti che in generale una complessità *Θ(degree(v))* è molto più favorevole rispetto sia a *Θ(n)* che a *Θ(m)*, anche perché di solito la stessa complessità va conteggiata su tutti i nodi, e la somma dei gradi di tutti i nodi di un grafo non orientato:

$$\sum_{i=1}^{n} degree(v_i)$$

è pari a *2m*, in quanto ogni singolo arco del grafo aumenta esattamente di 1 il grado di ciascuno dei due nodi in cui è incidente.

Se il grafo è orientato, una analoga considerazione può essere fatta per mostrare che:

- la somma dei gradi uscenti di tutti i nodi è pari a *m;*
- la somma dei gradi entranti di tutti i nodi è pari a *m*.

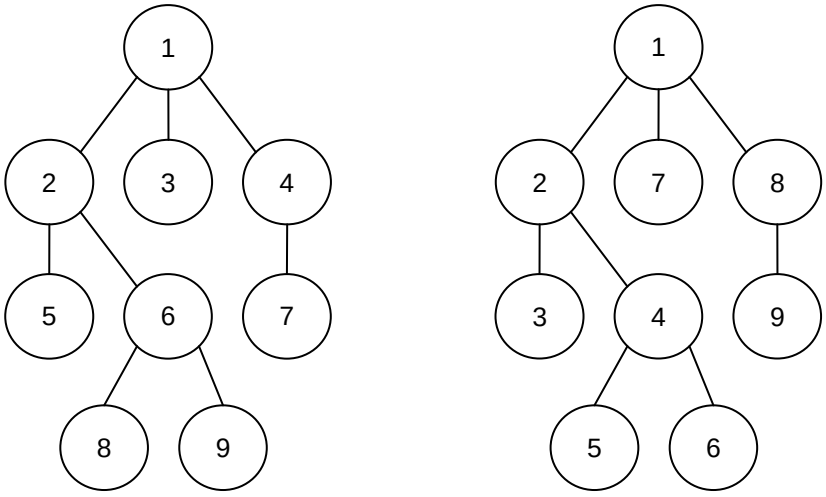
9.3 Visita di grafi

Analogamente a quanto già illustrato a proposito degli alberi, anche nel caso dei grafi un'operazione basilare, che spesso è il presupposto per poter risolvere il problema dato, è l'accesso sistematico a tutti i vertici, uno dopo l'altro, al fine di poter effettuare una specifica operazione (che dipende ovviamente dal problema posto) su ciascun vertice, ossia la ***visita del grafo***.

Partendo da uno specifico vertice, detto ***vertice di partenza***, e di solito identificato con *s*, si può desiderare di accedere a tutti i vertici che si trovano a una certa distanza da esso prima di accedere a quelli situati a distanza maggiore, e in questo caso si parla di ***visita in ampiezza (BFS, Breadth-first search)***; oppure si vuole accedere ai vertici allontandosi sempre di più dal vertice di partenza finché è possibile, e in questo caso si parla di ***visita in profondità (DFS, depth-first search)***.

Per capire di cosa stiamo parlando, ricordiamo che un albero è un grafo con certe proprietà (connesso e aciclico), e vediamo come vengono implementate le due visite ora menzionate su questi grafi molto semplici.

Nella figura seguente, i valori associati ai nodi indicano l'ordine col quale i nodi stessi vengono visitati. A sinistra è illustrato il percorso seguito dalla visita in ampiezza e a destra quello seguito dalla visita in profondità.



La visita in profondità è molto adatta ad essere realizzata in modo ricorsivo ma può, come vedremo, essere definita in modo iterativo con l'ausilio di una pila.

Viceversa, la visita in ampiezza è poco adatta a una implementazione ricorsiva e viene generalmente realizzata in modo iterativo con l'ausilio di una coda.

Queste due visite sono concettualmente piuttosto semplici però, nel momento in cui si passa dagli alberi ai grafi ci sono alcune complicazioni in più: dato che nei grafi possono essere presenti dei cicli (assenti negli alberi) la visita deve essere in grado di "ricordarsi" se un determinato vertice *v* è stato già visitato, poiché è possibile che diversi cammini che originano nel vertice di partenza conducano a *v*, e in tal caso il vertice *v* sarà "scoperto" più volte. Per questa ragione nelle visite di grafi è necessitano gestire alcune informazioni aggiuntive.

9.3.1 Alberi di visita

Indipendentemente dalla politica usata per determinare l'ordinamento nella scoperta di nuovi nodi, osserviamo che le visite hanno come prima conseguenza quella di verificare la connessione del grafo di partenza, infatti se partendo da un qualunque *s* si riescono a scoprire tutti gli *n* nodi non si può che concludere che il grafo è connesso. Viceversa, se si scopre solo un sottinsieme dei nodi, allora sapremo che quel sottinsieme induce la componente connessa contenente *s* e, per trovare le altre componenti, bisogna cominciare una nuova visita da un nodo non ancora visitato. Sarà necessario cominciare tante nuove visite quante sono le componenti connesse del grafo.

Mettiamoci ora nell'ipotesi che il grafo sia connesso e consideriamo il sottografo del grafo di partenza che ha come insieme di vertici l'intero insieme *V*, mentre l'insieme degli archi è costituito dall'insieme degli archi utilizzati dalla visita per scoprire nuovi nodi.

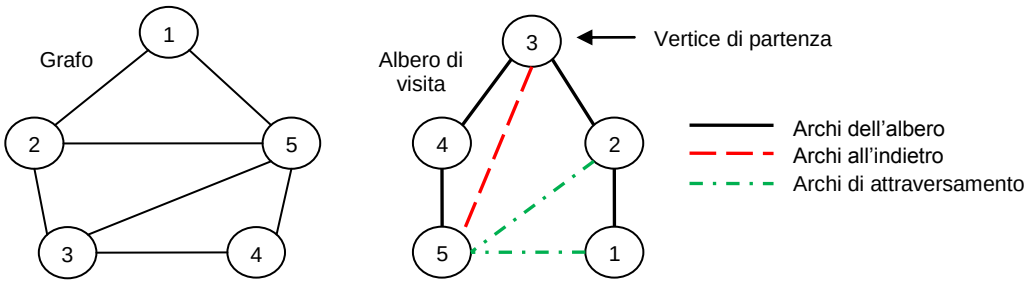
E' piuttosto semplice convincersi del fatto che tale sottografo è un albero. Infatti:

- è connesso in quanto, se *G* è connesso, allora ogni nodo è raggiungibile da *s* mediante un cammino, e questo risulterà durante la visita;
- è aciclico, infatti se per assurdo non lo fosse, potremmo ordinare i nodi che fanno parte del ciclo secondo la loro distanza da *s*; allora, il nodo più lontano da *s* risulterebbe avere due nodi dal quale è stato scoperto, e questo è ovviamente assurdo.

Una volta che ci siamo convinti che tale sottografo è un albero, possiamo osservare che è, in realtà, un **albero ricoprente,** poiché il suo insieme dei nodi coincide con quello dell'intero grafo.

Gli archi del grafo che fanno parte anche dell'albero ricoprente sono chiamati appunto **archi dell'albero**. Tutti gli altri archi, denominati genericamente **archi non dell'albero**, possono essere partizionati in due sottinsiemi: **archi all'indietro** e **archi di attraversamento**:

- gli archi all'indietro sono quelli che congiungono due nodi che sono uno l'antenato dell'altro
- gli archi di attraversamento sono tutti gli altri.



9.3.2 Visita in ampiezza (BFS)

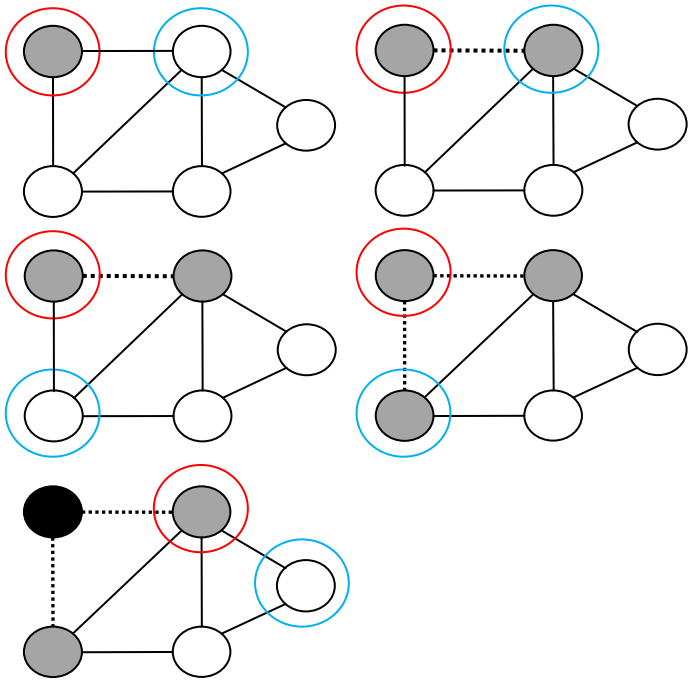
Dato un grafo *G* ed uno specifico ***vertice di partenza s***, la visita in ampiezza (BFS, Breadth-first search) si muove sistematicamente lungo gli archi di G per "scoprire", e successivamente visitare, ogni vertice *v* raggiungibile da *s*. Contestualmente costruisce un ***albero breadth-first*** (che inizialmente consiste del solo vertice s, la sua radice) che, alla fine, contiene tutti i vertici raggiungibili da *s*. Come vedremo più avanti, per ogni vertice *v* raggiungibile da *s* il cammino da *s* a *v* nell'albero breadth-first è un cammino minimo da *s* a *v*, ossia un cammino che contiene il minimo numero possibile di archi.

La visita in ampiezza espande la "frontiera" fra vertici già scoperti e vertici non ancora scoperti, allontanandola uniformemente e progressivamente dal vertice di partenza. In particolare, la

visita scopre tutti i vertici a distanza *k* dal vertice di partenza prima di scoprire anche un solo vertice a distanza *k + 1*.

Come accennato nel paragrafo precedente, è necessario tenere traccia dei vertici già visitati per evitare di visitarli più di una volta. A tale scopo i vertici vengono opportunamente ***marcati***.

Nel corso della scansione dei vertici adiacenti ad un vertice *u* già marcato (quindi già scoperto – vertice cerchiato di rosso nella figura), ogni volta che un nuovo vertice non marcato *v* viene scoperto (vertici cerchiati in azzurro nella figura), il vertice v viene marcato (reso grigio in figura) e l'arco (u, v) viene aggiunto all'albero breadth-first (arco tratteggiato nella figura). Non appena fra tutti gli adiacenti di *u* non vi è più alcun vertice bianco, il lavoro sul vertice *u* è terminato (*u* è reso nero in figura) e si prosegue passando ad un altro nodo marcato.



L'albero breadth-first si realizza mediante un campo aggiuntivo in ogni vertice, il campo ***predecessore***: aggiungere l'arco *(u, v)* all'albero breadth-first significa memorizzare nel campo *predecessore(v)* l'indice *u*. Poiché ogni vertice viene marcato solo una volta, ogni vertice avrà un solo predecessore e questo garantisce l'assenza di cicli nell'albero breadth-first: infatti, se vi fosse un ciclo, almeno un vertice del ciclo avrebbe due predecessori.

Pseudocodice

Lo pseudocodice riportato nel seguito assume che il grafo *G = (V, E)* in input sia rappresentato mediante liste di adiacenza.

Lo pseudocodice utilizza alcune informazioni aggiuntive, necessarie al suo corretto funzionamento:

- un campo *marked[v]* in ciascun vertice per gestire la marcatura dei vertici;
- un campo *pred[v]* in ciascun vertice per memorizzare il predecessore (inizialmente *NULL*);
- una coda Q per gestire l'ordine di visita dei vertici.

```
Funzione Breadth-first_search (G: grafo; q: puntatore a coda)
	Assegna a tutti i nodi v
		marked[v] ← false
		pred[v] ← NULL
	scegli il nodo di partenza s
	marked[s] ← true
	Enqueue(q, s)
	finché la coda Q non è vuota
		u ← Dequeue(q) // u è il vertice su cui lavorare
		visita il vertice u
		per ogni vertice v nella lista di adiacenza di u
			if marked[v] = false then
				marked[v] = true
				pred[v] ← u
				Enqueue(q, v)
	return
```

Complessità

L'inizializzazione ha complessità $O(|V|)$. Dopo tale fase nessun vertice viene più "smarcato", per cui ogni vertice viene marcato, inserito nella coda (e quindi estratto) al massimo una volta. Le operazioni di inserimento ed estrazione dalla coda hanno complessità $\Theta(1)$, quindi il tempo totale dedicato alle operazioni sulla coda è $O(|V|)$.

La lista di adiacenza di ciascun vertice è scorsa una volta sola, quando il vertice è estratto dalla coda. Poiché la somma delle lunghezze di tutte le liste di adiacenza è $\Theta(|E|)$, il tempo dedicato alle scansioni delle liste è al massimo $O(|E|)$.

Dunque la complessità totale è $O(|V| + |E|)$.

Proprietà della visita in ampiezza

La visita in ampiezza ha molte caratteristiche peculiari, che possono poi essere sfruttate per risolvere dei problemi opportuni, come vedremo.

Innanzi tutto, osserviamo che viene operata una politica di tipo FIFO (First In First Out) e questo viene confermato dall'utilizzo della coda.

Inoltre, l'albero di visita che scaturisce da una BFS ha una struttura partcolare.

Infatti si può osservare che, rispetto a tale albero, **nell'insieme degli archi non dell'albero non sono presenti archi all'indietro e gli archi di attraversamento collegano nodi sullo stesso livello o su due livelli adiacenti**. Per convincersi che fra gli archi non dell'albero non ci sono archi all'indietro, basta considerare il fatto che quando ci si trova su un nodo *u* si visitano tutti i suoi adiacenti non visitati, perciò non è possibile trovare nel sottoalbero di *u* un nodo *v*, scoperto quindi solo successivamente, che sia adiacente a *u*, perché altrimenti tale nodo sarebbe stato scoperto come adiacente di *u* e l'arco (u, v) farebbe parte dell'albero. Per un motivo analogo gli archi di attraversamento non possono collegare nodi che non siano sullo stesso livello o su livelli consecutivi: infatti se consideriamo due nodi *u* e *v* sui livelli *i* ed $i + j$, $j > 1$, rispettivamente, questi non possono in alcun modo essere collegati da un arco non dell'albero, perché allora il nodo *v* a livello $i + j$ sarebbe stato scoperto come figlio di *u* e l'arco (u, v) farebbe parte dell'albero.

Inoltre, **la visita in ampiezza individua i cammini più corti da *s* ad ogni altro nodo, e questi sono i cammini sull'albero**. Per convincersi di questo, si consideri un qualunque nodo *v*, ed il cammino più corto che collega *v* ad *s*. Tale cammino può essere costituito da archi dell'albero, archi di attraversamento tra nodi sullo stesso livello ed archi di attraversamento tra nodi su livelli consecutivi. Gli archi dell'albero e quelli di attraversamento tra nodi su livelli adiacenti portano da un livello *i* ad un livello $i + 1$, mentre gli archi di attraversamento che collegano nodi sullo stesso livello non fanno procedere verso livelli successivi. Ne consegue che questo ipotetico cammino più corto non potrà che essere della stessa lunghezza, o addirittura più lungo, del cammino che da *v* risale sull'albero verso *s*.

9.3.3 Visita in profondità

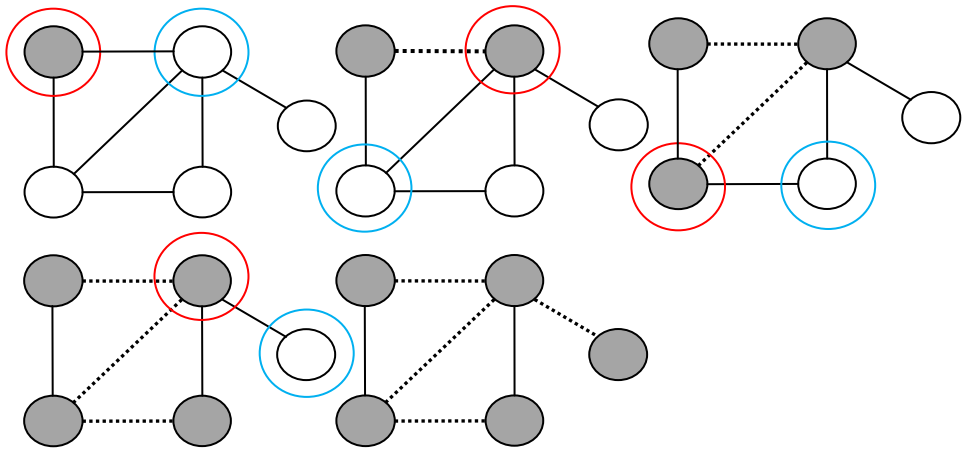
Dato un grafo *G* ed un vertice di partenza s, la visita in profondità (DFS, Depth-first search), a differenza di quella in ampiezza, esplora il grafo allontanandosi sempre più dal vertice di partenza finché è possibile, per poi ripartire da vertici più vicini a quello di partenza solo quando non è possibile allontanarsi ancora.

In particolare, se la visita in profondità muovendosi lungo l'arco *(u, v)* del grafo "scopre" il vertice *v*, proseguirà percorrendo uno degli archi uscenti da *v* e non percorrendo uno degli altri archi uscenti da *u*.

Contestualmente, la visita in profondità costruisce un ***albero depth-first*** che ha come radice il vertice *s* e contiene tutti i vertici raggiungibili da *s*.

Come nel caso della visita in ampiezza, è necessario tenere traccia dei vertici già visitati per evitare di visitarli più di una volta. A tale scopo i vertici vengono marcati, con una logica analoga a quella utilizzata nella visita in ampiezza, solamente la prima volta che vengono incontrati.

Il lavoro su un vertice *u* già marcato (quindi già scoperto – vertice cerchiato di rosso nella figura) consiste nella ricerca di un suo adiacente non marcato *v* (vertici cerchiati in azzurro nella figura). Il vertice v viene marcato (reso grigio in figura) e l'arco (u, v) viene aggiunto all'albero depth-first (arco tratteggiato nella figura). Il lavoro prosegue quindi sul vertice *v* anziché (come nella ricerca breadth-first) su un ulteriore adiacente di *u*. Si torna a lavorare su vertici precedenti solo quando non è più possibile proseguire il lavoro allontanandosi ulteriormente vertice di partenza.



Anche l'albero depth-first si realizza mediante un campo aggiuntivo in ogni vertice, il campo ***predecessore***: aggiungere l'arco (u, v) all'albero depth-first significa memorizzare nel campo

predecessore(v) l'indice *u*. Poiché ogni vertice viene marcato solo una volta, ogni vertice avrà un solo predecessore e questo garantisce l'assenza di cicli nell'albero depth-first.

Pseudocodice

La visita in profondità può essere espressa ricorsivamente in modo semplice e diretto, analogamente a quanto già discusso in relazione alle visite di alberi.

Lo pseudocodice riportato nel seguito assume che il grafo *G = (V, E)* in input sia rappresentato mediante liste di adiacenza. Le informazioni aggiuntive, necessarie al suo corretto funzionamento, sono le stesse già discusse per la visita in ampiezza:

- un campo *marked[v]* in ciascun vertice per gestire la marcatura dei vertici;
- un campo *pred[v]* in ciascun vertice per memorizzare il predecessore (inizialmente *NULL*).

Inoltre, lo pseudocodice presuppone che al momento della chiamata iniziale (alla quale si passa il vertice di partenza) tutti i nodi siano già inizializzati con il campo *marked* a *false* ed il campo *pred* a *NULL*.

```
Funzione Depth_first_search_ricorsiva (u: vertice)
        marked[u] ← true
        "visita u"
        "per ogni vertice v nella lista di adiacenza di u"
                if marked[v] = false then
                        pred[v] ← u
                        Depth_first_search_ricorsiva (v)
        return
```

Complessità

Per quanto riguarda la complessità della visita ricorsiva valgono le seguenti considerazioni:

- per ogni vertice v_i raggiungibile dal vertice di partenza vi è una e una sola chiamata ricorsiva, che avviene quando il vertice viene scoperto per la prima volta (e quindi non è ancora marcato): *O(n)*;
- nella chiamata ricorsiva relativa al vertice v_i si scandisce la lista dei suoi adiacenti: $O(degree(v_i))$.

Quindi la complessità totale è:

$$O(n) + \sum_{i=1}^{n} degree(v_i) = O(n) + O(m)$$

Proprietà della visita in profondità

Anche la visita in profondità ha delle interessanti proprietà.

Innanzi tutto è caratterizzata da una politica di tipo LIFO (Last In First Out), tanto è vero che, come vedremo, può essere realizzata con l'ausilio di una pila.

Anche l'albero di visita che scaturisce da una DFS ha una struttura partcolare.

Infatti si può osservare che, rispetto a tale albero, **nell'insieme degli archi non dell'albero non sono presenti archi di attraversamento**. Infatti, è assurdo pensare che esista un tale arco (u, v) poiché durante la visita in profondità il nodo v sarebbe trovato come discendente di u o viceversa (a seconda che la visita passi prima per u o prima per v) e quindi tale arco farebbe parte dell'albero.

Questa proprietà implica il seguente

Teorema.

Dato un grafo G, o G ha un cammino di lunghezza k o |E| = O(kn).

Dimostrazione.

Se *G* ha un cammino lungo *k* abbiamo finito, quindi supponiamo che ogni cammino sia di lunghezza < *k*. Limitiamo |*E*| contando il numero di antenati di ogni nodo, che corrisponde al suo massimo grado possibile. Infatti, non esistendo archi di attraversamento fra quelli non dell'albero, ogni nodo può essere collegato ad altri nodi esclusivamente da archi dell'albero ed archi all'indietro, ossia archi che lo collegano a propri antenati. Poiché tutti i cammini sono più corti di *k*, il numero degli antenati di ciascun nodo è < k, da cui segue che il numero di archi è minore di *kn*. CVD

9.3.4 Somiglianze fra la visita in ampiezza e la visita in profondità

Anche se non è molto evidente dallo pseudocodice illustrato nei due paragrafi precedenti, la visita in ampiezza e quella in profondità sono molto strettamente correlate fra loro.

In effetti, con un piccolo sforzo le due visite possono essere organizzate nello stesso identico modo, con l'unica differenza della struttura dati d'appoggio utilizzata: una coda nella visita in ampiezza ed una pila nella visita in profondità (le differenze sono evidenziate in grassetto sottolineato).

Perché sia possibile questa identità nella struttura dello pseudocodice è necessario inserire ed estrarre dalle strutture dati d'appoggio archi anziché nodi, altrimenti la visita in profondità non coincide funzionalmente con quella definita ricorsivamente.

Pseudocodice della visita in ampiezza

```
Funzione Breadth-first_search (G: grafo; q: puntatore a coda)
        Assegna a tutti i nodi v
                marked[v] ← false
                pred[v] ← NULL
        scegli il nodo di partenza s
        marked[s] ← true
        visita s
        per ogni vertice t nella lista di adiacenza di s
                Enqueue(q, arco(s, t))
        finché la coda Q non è vuota
                arco(x, y) ← Dequeue(q)
                if (marked[y] = false)
                        marked[y] = true
                        pred[y] ← x
                        visita(y)
                        per ogni vertice z nella lista di adiacenza di y
                                Enqueue(q, arco(y, z))
        return
```

Pseudocodice della visita in profondità

```
Funzione Depth-first_search (G: grafo; p: puntatore a pila)
        Assegna a tutti i nodi v
                marked[v] ← false
                pred[v] ← NULL
        scegli il nodo di partenza s
        marked[s] ← true
        visita s
        per ogni vertice t nella lista di adiacenza di s
                Push(p, arco(s, t))
        finché la pila P non è vuota
                arco(x, y) ← Pop(p)
                if (marked[y] = false)
                        marked[y] = true
                        pred[y] ← x
                        visita(y)
                        per ogni vertice z nella lista di adiacenza di y
                                Push(p, arco(y, z))
        return
```

Complessità di entrambe le visite

L'inizializzazione ha complessità $O(|V|) = O(n)$. Dopo tale fase nessun vertice viene più "smarcato", per cui ogni vertice viene marcato una sola volta, e ciò significa che, per ciascun vertice, uno solo degli archi in esso incidenti viene inserito nella (e quindi estratto dalla) struttura dati d'appoggio. Le operazioni di inserimento ed estrazione dalla struttura dati d'appoggio hanno complessità $\Theta(1)$, quindi il tempo totale dedicato alle operazioni sulla struttura dati d'appoggio è $O(n)$.

La lista di adiacenza di ciascun vertice è scorsa una volta sola, quando l'arco incidente nel vertice (arco che, per quanto detto sopra, è l'unico ad essere stato inserito) viene estratto dalla struttura dati d'appoggio.

La scansione delle liste di adiacenza però ha un costo che dipende dalla struttura dati utilizzata per l'implementazione. In particolare per un grafo con *n* nodi ed *m* archi:

- liste di adiacenza: per ogni vertice v_i si scandisce la lista dei suoi adiacenti, con un costo $O(degree(v_i))$, quindi la complessità totale è $O(n) + \sum_{i=1}^{n} degree(v_i) = O(n) + O(m)$;
- matrice di adiacenza: per ogni vertice v_i si scandisce la riga i-esima della matrice di adiacenza, con un costo $O(n)$, quindi la complessità totale è $O(n^2)$;
- matrice di incidenza: per ogni vertice v_i si scandisce la riga i-esima della matrice di incidenza, con un costo $O(m)$, quindi la complessità totale è $O(nm)$;
- lista di archi: per ogni vertice v_i si scandisce l'intero vettore contenente gli archi, con un costo $O(m)$, quindi la complessità totale è $O(nm)$.

Considerazioni del tutto analoghe si possono fare anche per le visite illustrate nei precedenti paragrafi 9.3.2 e 9.3.3.